TCD3

Test Automation & Continuous Delivery

Exkurs: GA & TSP

S. Wagner

Metaheuristiken

Metaheuristiken lassen sich einteilen in:

- **nachbarschaftsbasierte Metaheuristiken**
  - Local Search (LS)
  - Simulated Annealing (SA)
  - Tabu Search (TS)
  - …
- **populationsbasierte Metaheuristiken**
  - Ant Colony Optimization (ACO)
  - Particle Swarm Optimization (PSO)
  - evolutionäre Algorithmen (EA)
    - genetische Algorithmen (GA)
    - Evolutionsstrategien (ES)
    - genetische Programmierung (GP)
  - …

Evolutionäre Algorithmen

- Idee:
  Population von Lösungen entwickelt sich durch Selektion, Kreuzung und Mutation

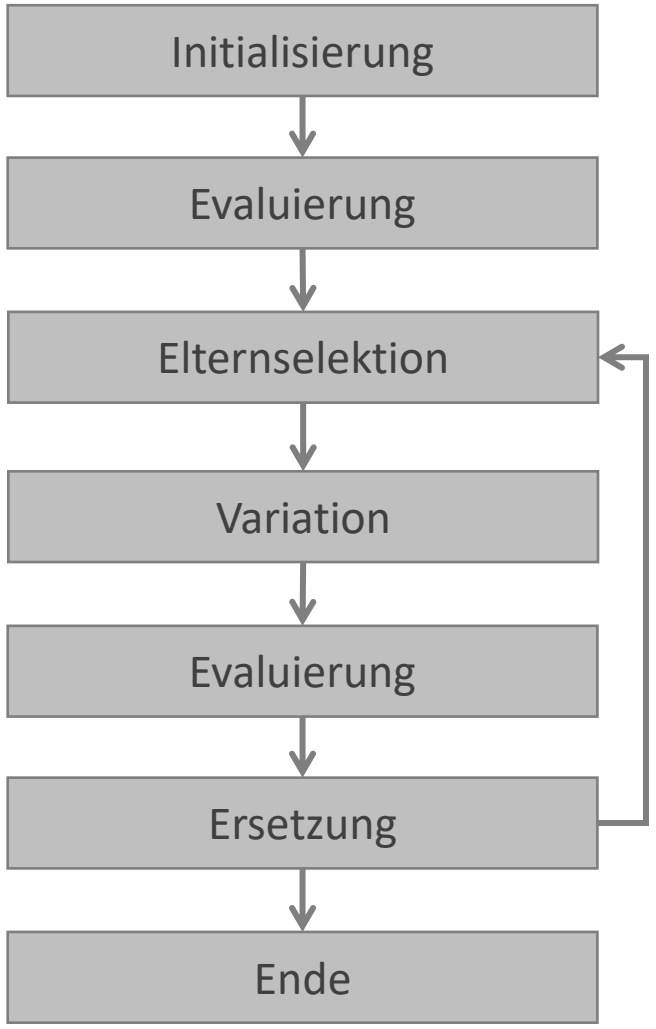
- Naturanalogie:
  Evolution der Arten, Survival of the Fittest (Darwin)
  Anpassung der Arten kann als Optimierungsprozess gesehen werden

- 3 Schritte:
  - **Selektion:** Auswahl guter Lösungen anhand der Lösungsqualität
  - **Variation:** Erzeugung neuer Lösungen durch Kreuzung und/oder Mutation
  - **Ersetzung:** Auswahl der nächsten Generation anhand der Lösungsqualität

- Bezeichnungen:
  - Lösungsqualität = Fitness
  - Qualitätsfunktion = Fitessfunktion
  - Lösung = Individuum
  - Lösungsrepräsentation = genetischer Code

Ablauf eines EA

- **Initialisierung:**
  initiale Population wird zufällig oder heuristisch erzeugt
- **Evaluierung:**
  Individuen (d.h. Lösungen) werden evaluiert, um ihre Fitness (d.h. Qualität) zu bestimmen
- **Elternselektion:**
  anhand der Fitness werden Individuen als Eltern für die nächste Generation ausgewählt
- **Variation:**
  durch Kreuzung und Mutation werden aus den Eltern neue Kind-Lösungen erzeugt
- **Ersetzung:**
  anhand der Fitness der neuen Kind-Lösungen wird bestimmt, welche Individuen die nächste Generation bilden



Genetischer Algorithmus (GA)

- Ablauf:
  1. erzeuge eine initiale Population von Lösungen
  2. evaluiere die Lösungen der initialen Population
  3. erzeuge eine neue Generation von Kindlösungen:
     a) selektiere Elternlösungen aus der aktuellen Population anhand der Fitness
     b) kreuze die Elternlösungen, um neue Kindlösungen zu erzeugen
     c) mutiere die Kindlösungen mit einer bestimmten Wahrscheinlichkeit
     d) evaluiere die Kindlösungen
  4. ersetze die alte Generation durch die neue Generation
  5. wiederhole ab Schritt 3 bis die Abbruchbedingung zutrifft

Genetischer Algorithmus (GA)

- Pseudocode:

```
GeneticAlgorithm(↓Problem p, ↓int popSize, ↓int maxGen, ↓double mutProb, ↑Solution best) {
  CreateInitialPopulation(↓Problem p, ↑Solution[1..popSize] pop)
  for (int i = 1..popSize) {
    Evaluate(↕Solution pop[i])
  }
  UpdateBest(↓Solution[1..popSize] pop, ↕Solution best)

  for (int j = 1..maxGen) {
    Solution[1..popSize] nextGen
    for (int k = 1..popSize) {
      Select(↓Solution[1..popSize] pop, ↑Solution mother)
      Select(↓Solution[1..popSize] pop, ↑Solution father)
      Cross(↓Solution mother, ↓Solution father, ↑Solution nextGen[k])
      Random(↑double random) // random number in [0,1[
      if (random < mutProb) {
        Mutate(↕Solution nextGen[k])
      }
      Evaluate(↕Solution nextGen[k])
    }
    Replace(↕Solution[1..popSize] pop, ↓Solution[1..popSize] nextGen)
    UpdateBest(↓Solution[1..popSize] pop, ↕Solution best)
  }
}
```

Elternselektion

- Auswahl überdurchschnittlich guter Individuen als Eltern für die nächste Generation
  - Selektion anhand der Fitness

- gängige Varianten:
  - Fitnessproportionale Selektion (Roulette Rad Selektion)
  - lineare Rangselektion
  - Turnierselektion

Gängige Varianten für die Elternselektion

- Turnierselektion (TS*n*)
  - zufällige Auswahl einer Turniergruppe
  - selektiert wird beste Lösung aus der Gruppe
  - Gruppengröße *n* beeinflusst Selektionswahrscheinlichkeit

$$P(s_i) = \frac{r(s_i)^n - (r(s_i) - 1)^n}{|Pop|^n}$$

  - Beispiel TS2 und TS4:

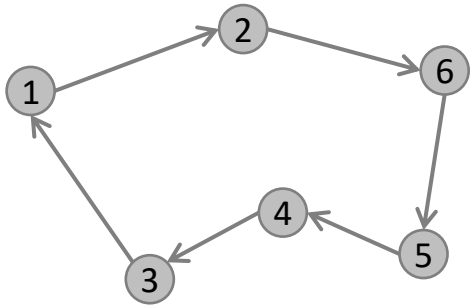
	Fitness $f(s_i)$	Rang $r(s_i)$	Fälle	**$P(s_i)$**
s_1	20 746	1	1	**4,00%**
s_2	21 333	2	3	**12,00%**
s_3	21 577	3	5	**20,00%**
s_4	22 689	4	7	**28,00%**
s_5	24 150	5	9	**36,00%**
Summe	110 495		**25**	
Gruppe *n*	**2**			

	Fitness $f(s_i)$	Rang $r(s_i)$	Fälle	**$P(s_i)$**
s_1	20 746	1	1	**0,16%**
s_2	21 333	2	15	**2,40%**
s_3	21 577	3	65	**10,40%**
s_4	22 689	4	175	**28,00%**
s_5	24 150	5	369	**59,04%**
Summe	110 495		**625**	
Gruppe *n*	**4**			

Lösungskodierungen für das TSP

- Matrixkodierung

	1	2	3	4	5	6
1	0	1	0	0	0	0
2	0	0	0	0	0	1
3	1	0	0	0	0	0
4	0	0	1	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	1	0



- Pfadkodierung:

1	2	6	5	4	3

- Adjazenzkodierung:

1	2	3	4	5	6
2	6	1	3	4	5

Lösungskodierungen für das TSP

Lösungskodierungen haben individuelle Stärken und Schwächen

- Eignung einer Lösungskodierung ist vom Algorithmus abhängig, der dann mit diesen Lösungen arbeitet muss
- Wahl einer guten (d.h. für das Verfahren gut passenden) Lösungskodierung ist wesentlich

Vor- und Nachteile der Lösungskodierungen für das TSP:

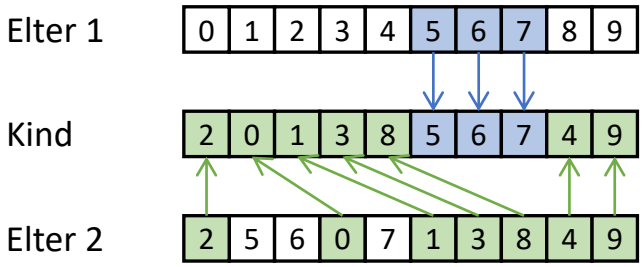
- **Matrixkodierung:**
  höherer Speicherbedarf, für manche Operationen sehr effizient
- **Pfadkodierung:**
  viele redundante Lösungen (d.h. verschiedene Permutationen kodieren eigentlich die gleiche Lösung), keine ungültigen Lösungen
- **Adjazenzkodierung:**
  kann auch ungültige Lösungen darstellen, dafür weniger redundante Lösungen wie bei der Pfaddarstellung

Kreuzung und Mutation

- Kreuzung und Mutation werden als genetische Operatoren bezeichnet
  - dienen dazu, Erbgut zu variieren, um neue Lösungen zu erzeugen
  - Kreuzung: Rekombination des Erbguts der Eltern
  - Mutation: geringfügige Änderung zur Variation einer Lösung (vgl. Nachbarschaftsfunktion bei nachbarschaftsbasierten Metaheuristiken)
- mögliche Operatoren hängen von der Lösungskodierung ab
- Beispiele für Binärkodierung
  - Kreuzung: Single Point Crossover (SPX), Multiple Point Crossover (MPX), Uniform Crossover (UX), …
  - Mutation: Bit Flip, Swap, …
- Beispiele für Permutationskodierung
  - Kreuzung: Order Crossover (OX), Cyclic Crossover (CX), …
  - Mutation: 2-opt, 3-opt, Swap, …

Kreuzungsoperatoren für Permutationen

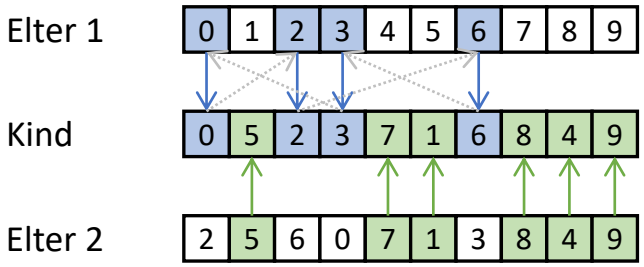
- Order Crossover (OX)
  - übernimm zufälliges Segment von einem Elternteil (1. Schritt)
  - fülle verbleibende Positionen in der Reihenfolge auf, wie sie im anderen Elternteil vorkommen (2. Schritt)



  - wenig positionserhaltend, Reihenfolge stärker beachtet

Kreuzungsoperatoren für Permutationen

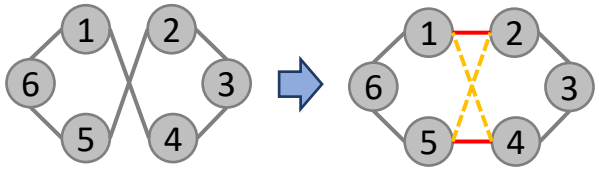
- Cyclic Crossover (CX)
  - übernimm eine zufällige Position aus dem ersten Elternteil (im Beispiel "0")
  - übernimm den Wert aus dem ersten Elternteil, der an dieser Position im anderen Elternteil steht (im Beispiel "2")
  - setze fort, bis sich ein Zyklus ergibt (1. Schritt)
  - übernimm die restlichen Positionen aus dem anderen Elternteil (2. Schritt)



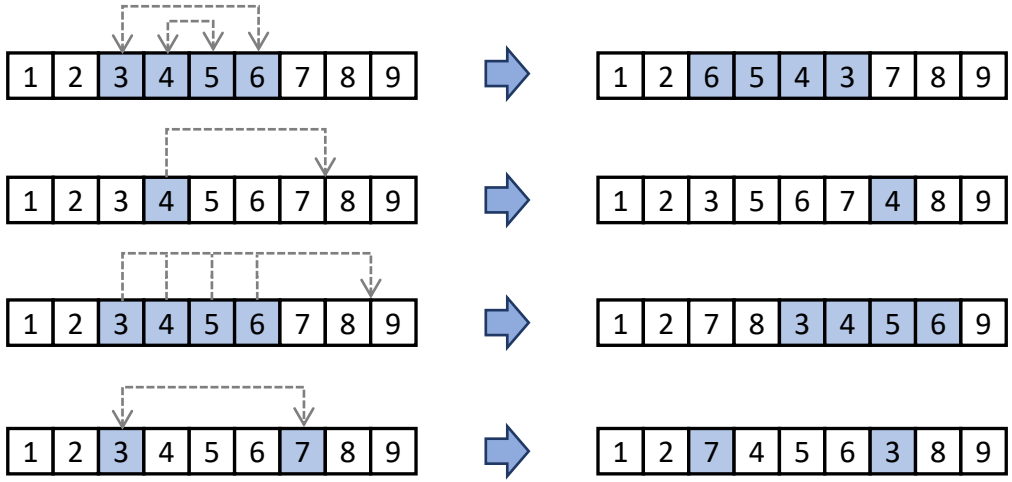
  - ausschließlich positionserhaltend

Mutationsoperatoren für Permutationen

- *k*-opt Mutation: Austausch von *k* Kanten
  - z.B. 2-opt



- Mutationen bei Pfadkodierung:
  - Inversion (2-opt)
  - Insertion (3-opt)
  - Translocation (3-opt)
  - Swap2 (4-opt)



Ersetzung

- bestimmt welche Lösungen als nächste Generation ausgewählt werden
- Auswahl von Lösungen aus der alten Generation und aus den erzeugten Kindern

- gängige Varianten:
  - **Generational Replacement:**
    Kinder ersetzen die Elterngeneration vollständig
    beste Lösung der aktuellen Generation kann sich auch wieder verschlechtern (Lösungsverlauf ist nicht monoton)
  - ***n*-Elitismus:**
    n besten Lösungen aus der Elterngeneration werden in die nächste Generation übernommen, der Rest wird mit den besten Kindern aufgefüllt (üblichste Variante ist 1-Elitismus)
    beste Lösung der aktuellen Generation ist immer die insgesamt beste bis jetzt gefundene Lösung (Lösungsverlauf ist monoton)
  - **Steady-State-Replacement:**
    in jeder Iteration wird nur eine Lösung aus der Elterngeneration durch eine Kindlösung ersetzt